DATA SCIENCE AND MACHINE LEARNING

Himanshu Singh
himanshusingh1814@gmail.com

8-Week Python for Data Science Training Module

This curriculum assumes basic Python familiarity (variables, loops, functions, basic data structures) and dives directly into the data science stack.

- **PYTHON Basics.**
- **Python Data structures.**
- **Python modules.**
- **Python OOPS.**
- **Python RegEx.**
- **Python file handling.**
- **Numpy**
- **Pandas**
- **Scipy**
- **Matplotlib**

Machine Learning

- Getting Started
- Mean Median Mode
- Standard Deviation
- Percentile
- Data Distribution
- Normal Data Distribution
- Scatter Plot
- Linear Regression
- Polynomial Regression
- Multiple Regression
- ScaleTrain/Test
- Decision Tree
- Confusion Matrix
- Hierarchical Clustering
- Logistic Regression
- Grid Search
- Categorical DataK-means
- Bootstrap Aggregation
- Cross Validation
- AUC - ROC Curve
- K-nearest neighbors

NumPy Tutorial

NumPy Random

NumPy ufunc

Pandas Tutorial

Cleaning Data

Correlations

Plotting

Pandas Plotting

R Tutorial

R Data Structures

R Graphics

R Statistics

SciPy Tutorial

Data Science

DS Math

DS Statistics

DS Advanced

Week 1: Data Manipulation with NumPy

Learning Objectives

- Understand ndarray architecture and memory layout
- Perform vectorized operations and broadcasting
- Apply advanced indexing and slicing techniques

Topics

- Array creation: np.array(), np.zeros(), np.ones(), np.arange(), np.linspace(), np.random
- Array attributes: shape, dtype, ndim, strides
- Reshaping and transposing: reshape(), ravel(), flatten(), .T, np.swapaxes()
- Broadcasting rules and avoiding shape mismatches
- Boolean and fancy indexing for conditional selection
- Aggregations: sum(), mean(), std(), min(), max() along axes
- Linear algebra basics: np.dot(), np.matmul(), @ operator, np.linalg

Hands-On Project Build a basic image processor that loads an image as a NumPy array, applies transformations (grayscale conversion, brightness adjustment, cropping), and outputs the result.

Week 2: Data Wrangling with Pandas (Part 1)

Learning Objectives

- Master DataFrame and Series structures
- Perform data loading, inspection, and basic cleaning
- Handle missing data effectively

Topics

- Series and DataFrame creation from dicts, lists, NumPy arrays
- Reading data: read_csv(), read_excel(), read_json(), read_sql()
- Inspection methods: head(), tail(), info(), describe(), dtypes, shape
- Selection: loc[], iloc[], boolean indexing, query()
- Missing data: isna(), fillna(), dropna(), interpolation strategies
- Data type conversion: astype(), pd.to_datetime(), pd.to_numeric()
- String operations: .str accessor for cleaning text columns

Hands-On Project Clean a messy real-world dataset (e.g., Kaggle's Titanic or a public census dataset)–handle missing values, fix dtypes, standardize text fields, and export a clean CSV.

Week 3: Data Wrangling with Pandas (Part 2)

Learning Objectives

- Perform complex transformations and aggregations
- Merge, join, and reshape datasets
- Optimize performance for large datasets

Topics

- GroupBy operations: groupby(), aggregation functions, transform(), apply()
- Pivot tables and cross-tabulations: pivot_table(), crosstab(), melt(), stack(), unstack()
- Merging and joining: merge(), join(), concat(), handling duplicates
- Window functions: rolling(), expanding(), shift(), diff()
- Categorical data: pd.Categorical, memory optimization, ordered categories
- Performance tips: avoiding chained indexing, eval(), query(), chunked reading

Hands-On Project Analyze a multi-table e-commerce dataset–join orders, customers, and products tables, compute monthly revenue by category, and identify top customers using window functions.

Week 4: Data Visualization

Learning Objectives

- Create publication-quality static visualizations
- Build interactive dashboards
- Choose appropriate chart types for different data stories

Topics

- Matplotlib fundamentals: figure/axes architecture, subplots, customization (colors, labels, legends, annotations)
- Seaborn for statistical graphics: relplot(), catplot(), displot(), heatmap(), pairplot()
- Plotly for interactivity: line, bar, scatter, geographic maps, plotly.express vs graph_objects
- Customization: themes, color palettes, faceting, dual axes
- Best practices: avoiding chartjunk, accessibility considerations, storytelling with data

Hands-On Project Create a visual analytics report on a dataset of your choice–include at least 5 different chart types, one interactive Plotly dashboard, and a narrative explaining insights.

Week 5: Exploratory Data Analysis & Feature Engineering

Learning Objectives

- Conduct systematic EDA workflows
- Engineer features that improve model performance
- Detect and handle outliers and anomalies

Topics

- EDA workflow: univariate → bivariate → multivariate analysis
- Distribution analysis: histograms, KDE plots, Q-Q plots, skewness, kurtosis
- Correlation analysis: Pearson, Spearman, corr(), heatmaps, multicollinearity detection
- Outlier detection: IQR method, Z-scores, isolation forests
- Feature engineering techniques:
    - Binning and discretization
    - Log/power transformations
    - Date/time feature extraction
    - Text feature extraction (TF-IDF basics)
    - Target encoding, one-hot encoding, label encoding
- Feature selection: variance threshold, correlation filtering, mutual information

Hands-On Project Perform comprehensive EDA on a housing prices dataset, engineer at least 10 new features, and document findings in a Jupyter notebook with visualizations and insights.

Week 6: Machine Learning with Scikit-Learn

Learning Objectives

- Understand the scikit-learn API and workflow
- Implement classification, regression, and clustering algorithms
- Evaluate and compare model performance

Topics

- Scikit-learn API: estimators, transformers, predictors, pipelines
- Preprocessing: StandardScaler, MinMaxScaler, OneHotEncoder, LabelEncoder, SimpleImputer
- Train-test split and cross-validation: train_test_split(), cross_val_score(), KFold, StratifiedKFold
- Supervised learning:
    - Regression: Linear Regression, Ridge, Lasso, Random Forest Regressor, Gradient Boosting
    - Classification: Logistic Regression, Decision Trees, Random Forest, SVM, k-NN
- Unsupervised learning: K-Means, DBSCAN, hierarchical clustering, PCA, t-SNE
- Evaluation metrics:
    - Regression: MSE, RMSE, MAE, R^2

- Classification: accuracy, precision, recall, F1, ROC-AUC, confusion matrix
- Pipelines: Pipeline, ColumnTransformer, make_pipeline()

Hands-On Project Build an end-to-end ML pipeline for a classification problem (e.g., customer churn, loan default)–from preprocessing through model comparison to final evaluation.

Week 7: Advanced ML & Model Optimization

Learning Objectives

- Tune hyperparameters systematically
- Handle imbalanced datasets
- Deploy models using serialization

Topics

- Hyperparameter tuning: GridSearchCV, RandomizedSearchCV, Bayesian optimization (Optuna intro)
- Ensemble methods deep dive: bagging vs boosting, XGBoost, LightGBM, CatBoost
- Handling imbalanced data: SMOTE, class weights, threshold tuning, stratified sampling
- Model interpretation: feature importances, permutation importance, SHAP values
- Model persistence: joblib, pickle, versioning best practices
- Introduction to AutoML: TPOT, auto-sklearn concepts

Hands-On Project Take the Week 6 model, optimize it using hyperparameter tuning, apply SHAP for interpretation, and serialize the final model with a simple prediction script.

Week 8: Real-World Data Science Workflows

Learning Objectives

- Work with SQL databases from Python
- Handle time series data
- Structure reproducible data science projects

Topics

- SQL integration: sqlite3, sqlalchemy, pandas.read_sql(), parameterized queries
- Time series analysis:
  - DateTime indexing in Pandas
  - Resampling, rolling statistics, seasonal decomposition
  - Basic forecasting: ARIMA concepts, Prophet introduction
- Working with APIs: requests, JSON parsing, pagination handling
- Project structure: cookiecutter data science, virtual environments, requirements.txt
- Version control for data science: Git basics, .gitignore for data files, DVC introduction

- Reproducibility: random seeds, environment management (conda, pip-tools), documentation

Capstone Project End-to-end data science project:

1. Collect data from an API or public source
2. Store in SQLite database
3. Perform EDA and feature engineering
4. Build and optimize ML model
5. Create visualization dashboard
6. Document in a well-structured repository with README